

UNIVERSIDAD AUTONOMA DE OCCIDENTE

UAO Virtual

Especializacion en Inteligencia Artificial

MICROPROYECTO 1

Cluster LXD + HAProxy con Servidores Backup

Estudiante:	Milton Jaramillo
Docente:	Prof. Heberth Fabian Martinez Velasquez
Materia:	Computacion en la Nube
Programa:	Especializacion en Inteligencia Artificial
Institucion:	UAO Virtual — Universidad Autonoma de Occidente
Fecha:	Mayo 2026

1. Introduccion

La computacion en la nube exige arquitecturas que garanticen alta disponibilidad, escalabilidad y tolerancia a fallos. En entornos reales de produccion, no basta con un simple balanceador de carga: se requiere una infraestructura distribuida capaz de mantener el servicio operativo incluso cuando multiples nodos fallan de manera simultanea. Este microproyecto aborda precisamente ese desafio, escalando la practica anterior de HAProxy simple hacia una arquitectura mas robusta y realista.

El proyecto implementa un cluster LXD multi-nodo sobre las dos VMs Vagrant existentes (servidorUbuntu y clienteUbuntu), distribuyendo cinco contenedores entre ambos nodos. HAProxy se configura con una jerarquia de servidores: web1 y web2 como nodos de produccion activos con balanceo Round Robin, y web3 y web4 en clienteUbuntu como servidores de backup que se activan automaticamente cuando la produccion falla, sin ninguna intervencion manual.

Adicionalmente, se implementan paginas web con identidad visual institucional de la UAO, una pagina de error 503 personalizada, y se resuelve el reto tecnico de la comunicacion cross-node mediante proxy devices LXD. El microproyecto cierra con pruebas de carga comparativas usando Apache JMeter bajo dos escenarios: operacion normal y condiciones de estres extremo.

2. Objetivos

2.1 Objetivo General

Implementar una arquitectura de alta disponibilidad con cluster LXD multi-nodo y HAProxy configurado con servidores de produccion y backup, evaluando su comportamiento ante fallos en cascada y bajo carga concurrente con Apache JMeter.

2.2 Objetivos Especificos

- Configurar un cluster LXD de dos nodos usando las VMs Vagrant existentes mediante inicializacion con preseed y union por certificado.
- Distribuir cinco contenedores entre los dos nodos del cluster: web1, web2 y haproxy en servidorUbuntu (produccion), y web3 y web4 en clienteUbuntu (backup).
- Configurar HAProxy con algoritmo Round Robin para web1/web2 como activos y web3/web4 como servidores de backup con activacion automatica ante fallos.
- Resolver el reto de comunicacion cross-node entre el contenedor haproxy y los contenedores web3/web4 alojados en el nodo clienteUbuntu mediante proxy devices LXD.
- Disenar e implementar paginas HTML con identidad visual UAO diferenciadas por rol: verde para produccion y naranja para backup, incluyendo una pagina de error 503 personalizada.

- Ejecutar y comparar pruebas de carga JMeter en escenario normal (100 usuarios, 1,000 peticiones) y escenario de estres (500 usuarios, 11,000 peticiones).

3. Marco Teorico

3.1 Cluster LXD Multi-nodo

LXD soporta la operacion en modo cluster, permitiendo que multiples instancias de LXD en diferentes hosts actuen como un sistema unificado. En un cluster LXD, los contenedores pueden desplegarse en cualquier nodo y son visibles desde cualquier miembro a traves de la API compartida. El nodo inicial se configura como bootstrap con `lxd init --preseed`, definiendo la direccion del cluster, el certificado compartido y la clave de acceso. Los nodos adicionales se unen al cluster aportando su propio certificado de servidor y autenticandose con la contraseña del cluster. Esta arquitectura permite distribuir la carga computacional entre multiples maquinas fisicas o virtuales, aumentando la capacidad total del sistema.

3.2 HAProxy y Servidores Backup

HAProxy permite definir servidores de backup en el bloque backend mediante la directiva `backup`. Un servidor marcado como backup permanece inactivo mientras al menos un servidor activo este disponible. Cuando todos los servidores activos fallan sus health checks, HAProxy activa automaticamente los servidores backup sin requerir intervencion del administrador. Esta configuracion implementa el patron de failover automatico, fundamental en arquitecturas de alta disponibilidad. La directiva `option httpchk` permite configurar health checks HTTP mas sofisticados que los checks TCP basicos, verificando que el servidor no solo este accesible sino que responda correctamente a peticiones HTTP.

3.3 Comunicacion Cross-node en LXD

En un cluster LXD, los contenedores alojados en diferentes nodos no comparten la misma red bridge interna. Un contenedor en el nodo A no puede alcanzar directamente por IP a un contenedor en el nodo B. La solucion mediante proxy devices LXD consiste en crear un dispositivo de tipo proxy en el nodo B que escuche en una IP accesible desde el nodo A (la IP de la VM anfitriona) y redirija el trafico al contenedor destino. Esto crea un tunel de reenvio de puertos que permite la comunicacion inter-nodo sin modificar la configuracion de red del cluster.

4. Desarrollo del Microproyecto

4.1 Configuracion del Cluster LXD

El cluster se inicializo en servidorUbuntu como nodo bootstrap mediante un archivo `preseed` que define los parametros del cluster: nombre del nodo, direccion de escucha (192.168.100.3:8443), direccion del cluster y certificado compartido. El comando utilizado fue:

```
cat <<'EOF' | lxd init --preseed
cluster:
```

En clienteUbuntu, el nodo secundario se uno al cluster proporcionando el certificado del nodo bootstrap y la contraseña. Se verifico el estado del cluster con `lxc cluster list`, confirmando que ambos nodos aparecian en estado ONLINE y Fully operational.

```
vagrant@clienteUbuntu: ~$ cat /var/lib/lxd/cluster.crt: No such file or directory
vagrant@clienteUbuntu:~$ cat <<'EOF' | lxd init --preseed
cluster:
  server_name: clienteUbuntu
  enabled: true
  server_address: 192.168.100.2:8443
  cluster_address: 192.168.100.3:8443
  cluster_password: password123
  cluster_certificate: |
    -----BEGIN CERTIFICATE-----
    MIICHDCCAAGwAIBAgIQbodP3kmmELePI4YjY32ABDAKBgghkJPQQDZA8MRww
    GgYVQQKEKxSna5S1eGlvbnRhaW5lcnNub3JnMnRwGgYVQVQDBNb290QHNlcnZp
    ZG9yVWJ1bnRlMw4XDTI2MDUzMTIwMTA0NlOXDTM2MDUyODIwMTA0NlOWPDEcMBQg
    AUECHTBTBgUdXhjb250YVJlZXZlbnZyZCMBGGAUEAwmc9vdeBzZXZ2aARv
    c1VlMw50bDZlM2BgAGBvcm50aG9lZABCAzBCGACGPDCTCm9pOGEzKno5
    VH/anakJUI0tuzhiFxF/gLnaAde8AEYlW4f5LkE5BHRbQJ1C15SPHb1Q1j1+LcL91
    yL5oB13M98Q7K0jGkExhcyv29+XuPQd5vmsY6S5eAaNoMGwYdYVRB8Cw/BAQD
    AgwMBMGA1UdJQQMAAGCCGAQUFBwMBAwGA1UdEwEw/wQCMAAwMQYVDRRB8Cw
    KTI0C2Yydm1kb3J3YVJlVudHBBH8AAAGHEAAAAAAsAAAAAAAAAAAEwCgYIKoZI
    zj0EAMDAQAwZG1wAKCpdVakXlqzhjyXw+V1SxJ7HzWVslVhWpbxmbB1XsdnzUx
    ocB5GPZGKH8Yt1+JwIXAPE0ZVqgMlp3Q9w9F57+ccqyTn50QvzJfM69EzlujpJ
    M23ipSzLFAiK5yPeHb/TSQ==
    -----END CERTIFICATE-----
EOF

vagrant@clienteUbuntu:~$
vagrant@clienteUbuntu:~$ lxc cluster list
To start your first container, try: lxc launch ubuntu:20.04
Or for a virtual machine: lxc launch ubuntu:20.04 --vm
```

NAME	URL	DATABASE	STATE	MESSAGE
clienteUbuntu	https://192.168.100.2:8443	YES	ONLINE	Fully operation
x86_64				
servidorUbuntu	https://192.168.100.3:8443	YES	ONLINE	Fully operation
x86_64				

```
vagrant@clienteUbuntu:~$
vagrant@servidorUbuntu:~$ cat /var/lib/lxd/cluster.crt
cat: /var/lib/lxd/cluster.crt: No such file or directory
vagrant@servidorUbuntu:~$ find /var/lib/lxd /var/snap/lxd -name '*.crt' 2>/dev/null
vagrant@servidorUbuntu:~$ sudo find -name 'cluster.crt' 2>/dev/null && lxc info | head -20
/var/snap/lxd/common/lxd/cluster.crt
config:
  cluster_https_address: 192.168.100.3:8443
  core_https_address: 192.168.100.3:8443
  core_trust_password: true
api_extensions:
  storage_zfs_remove_snapshots
  container_host_shutdown_timeout
  container_stop_priority
  container_syscall_filtering
  auth_pki
  container_last_used_at
  etag
  patch
  usb_devices
  https_allowed_credentials
  image_compression_algorithm
  directory_manipulation
  container_cpu_time
  storage_zfs_use_refquota
  storage_lvm_mount_options
vagrant@servidorUbuntu:~$ cat /var/snap/lxd/common/lxd/cluster.crt
-----BEGIN CERTIFICATE-----
MIICHDCCAAGwAIBAgIQbodP3kmmELePI4YjY32ABDAKBgghkJPQQDZA8MRww
GgYVQQKEKxSna5S1eGlvbnRhaW5lcnNub3JnMnRwGgYVQVQDBNb290QHNlcnZp
ZG9yVWJ1bnRlMw4XDTI2MDUzMTIwMTA0NlOXDTM2MDUyODIwMTA0NlOWPDEcMBQg
AUECHTBTBgUdXhjb250YVJlZXZlbnZyZCMBGGAUEAwmc9vdeBzZXZ2aARv
c1VlMw50bDZlM2BgAGBvcm50aG9lZABCAzBCGACGPDCTCm9pOGEzKno5
VH/anakJUI0tuzhiFxF/gLnaAde8AEYlW4f5LkE5BHRbQJ1C15SPHb1Q1j1+LcL91
yL5oB13M98Q7K0jGkExhcyv29+XuPQd5vmsY6S5eAaNoMGwYdYVRB8Cw/BAQD
AgwMBMGA1UdJQQMAAGCCGAQUFBwMBAwGA1UdEwEw/wQCMAAwMQYVDRRB8Cw
KTI0C2Yydm1kb3J3YVJlVudHBBH8AAAGHEAAAAAAsAAAAAAAAAAAEwCgYIKoZI
zj0EAMDAQAwZG1wAKCpdVakXlqzhjyXw+V1SxJ7HzWVslVhWpbxmbB1XsdnzUx
ocB5GPZGKH8Yt1+JwIXAPE0ZVqgMlp3Q9w9F57+ccqyTn50QvzJfM69EzlujpJ
M23ipSzLFAiK5yPeHb/TSQ==
-----END CERTIFICATE-----
vagrant@servidorUbuntu:~$
```

4.2 Creacion y Distribucion de Contenedores

```
lxc launch ubuntu:20.04 web1 --target servidorUbuntu
lxc launch ubuntu:20.04 web2 --target servidorUbuntu
lxc launch ubuntu:20.04 web3 --target clienteUbuntu
lxc launch ubuntu:20.04 web4 --target clienteUbuntu
lxc launch ubuntu:20.04 haproxy --target servidorUbuntu
```

El comando `lxc list` mostro los cinco contenedores en estado `RUNNING` con sus IPs asignadas y la columna `LOCATION` indicando el nodo anfitrión de cada uno.

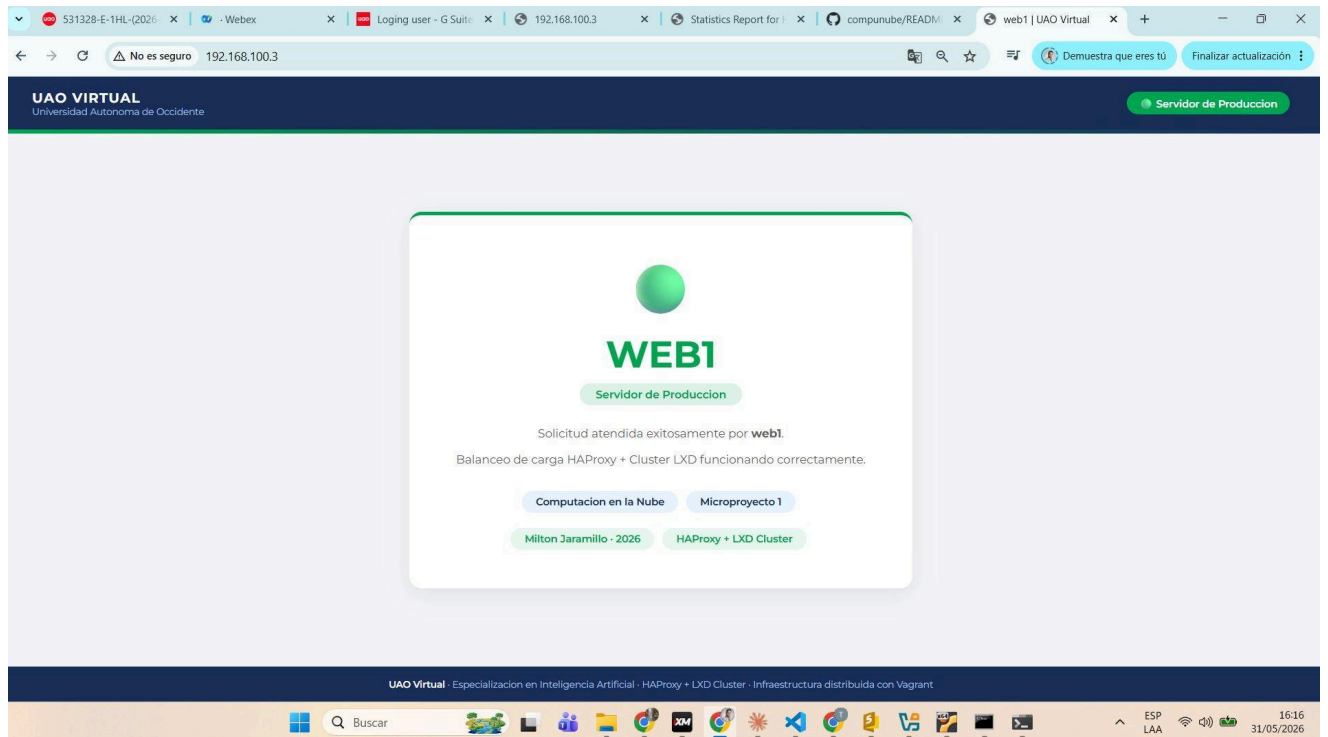


Figura 3. Pagina web1 con identidad UAO — Servidor de Produccion (verde)

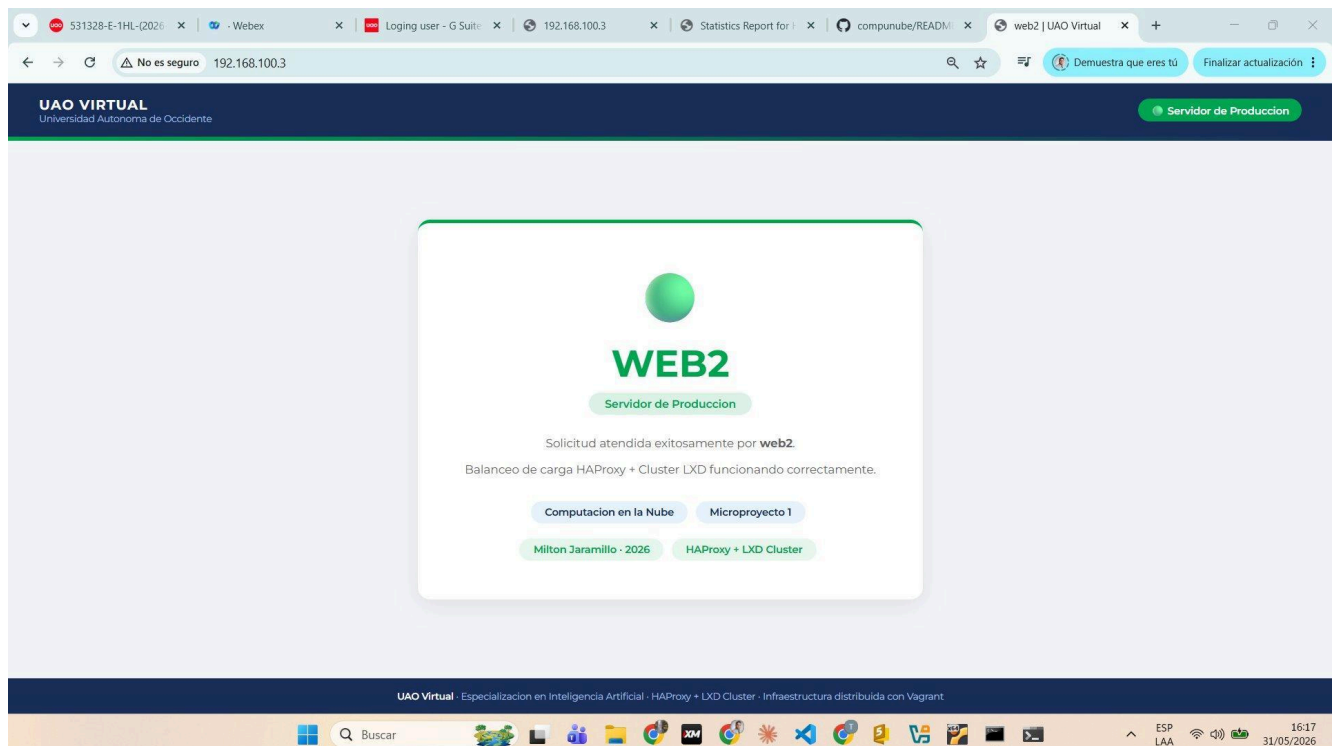


Figura 4. Pagina web2 con identidad UAO — Servidor de Produccion (verde)

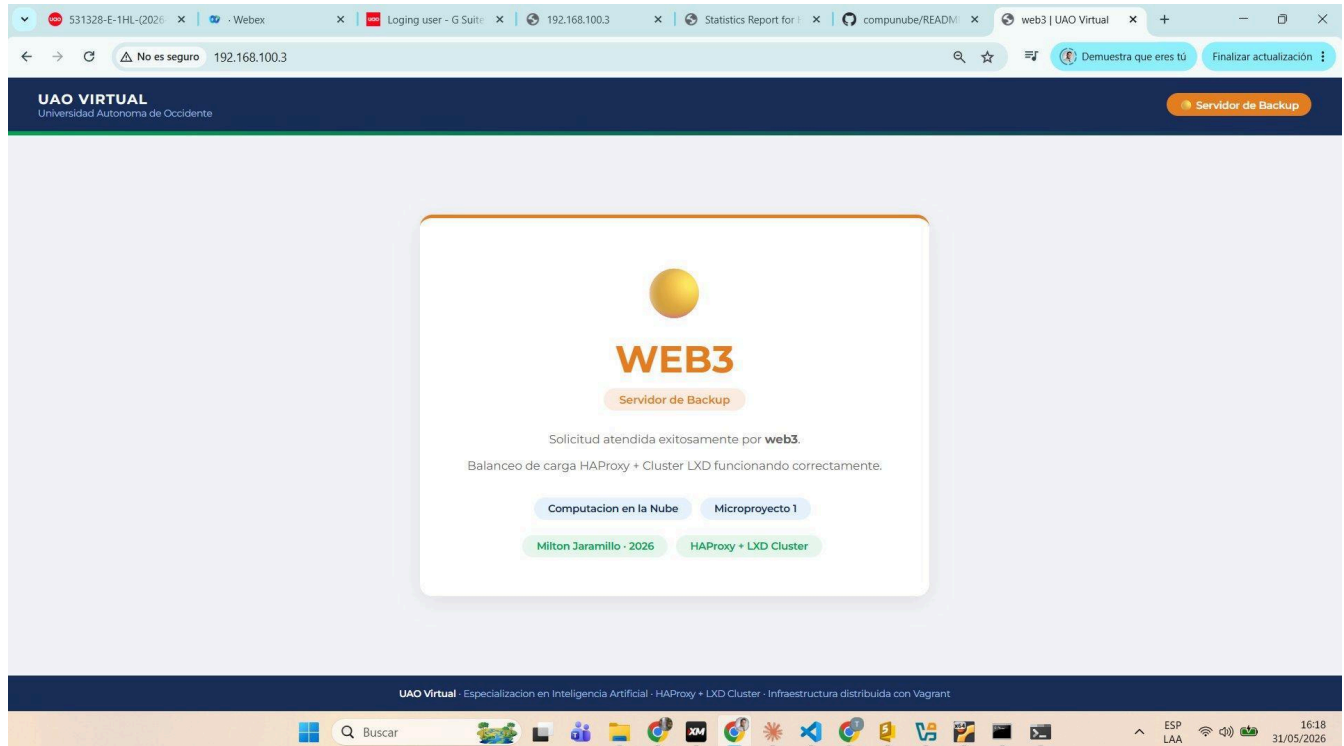


Figura 5. Pagina web3 con identidad UAO — Servidor de Backup (naranja)

4.4 Resolucion del Reto Cross-node

El contenedor haproxy en servidorUbuntu no podia alcanzar directamente las IPs internas de web3 y web4 en clienteUbuntu, ya que pertenecen a redes bridge diferentes en cada nodo. La solucion fue crear proxy devices en clienteUbuntu que exponen los contenedores de backup en la IP de la VM anfitriona (192.168.100.2) en puertos dedicados:

```
# En clienteUbuntu: exponer web3 en el puerto 8083
lxc config device add web3 http proxy listen=tcp:192.168.100.2:8083
connect=tcp:127.0.0.1:80
# En clienteUbuntu: exponer web4 en el puerto 8084
lxc config device add web4 http proxy listen=tcp:192.168.100.2:8084
connect=tcp:127.0.0.1:80
```

Con esto, HAProxy pudo referenciar los servidores backup como 192.168.100.2:8083 y 192.168.100.2:8084, que son accesibles desde cualquier contenedor en servidorUbuntu a traves de la red host-only de Vagrant.

4.5 Configuracion de HAProxy con Backup

El archivo /etc/haproxy/haproxy.cfg se configuro con dos grupos diferenciados: servidores activos con Round Robin y servidores de backup que se activan automaticamente. La seccion backend relevante fue:

```
backend web-backend
    balance roundrobin
    option httpchk GET /
    server web1 10.149.16.104:80 check
    server web2 10.149.16.49:80 check
```

```
server web3 192.168.100.2:8083 check backup
server web4 192.168.100.2:8084 check backup
errorfile 503 /etc/haproxy/errors/503.http
```

Se habilito tambien el panel de estadisticas en /haproxy?stats y se configuro una pagina de error 503 personalizada con la identidad visual UAO para el escenario en que todos los servidores, incluyendo los backup, esten caidos.

4.6 Verificacion del Dashboard HAProxy

Con todos los contenedores activos, el dashboard de HAProxy mostro web1 y web2 en verde (activos, estado UP) y web3 y web4 en azul (backup UP en espera). El health check L4OK confirmaba la conectividad correcta con todos los nodos, incluyendo los cross-node accedidos via proxy.

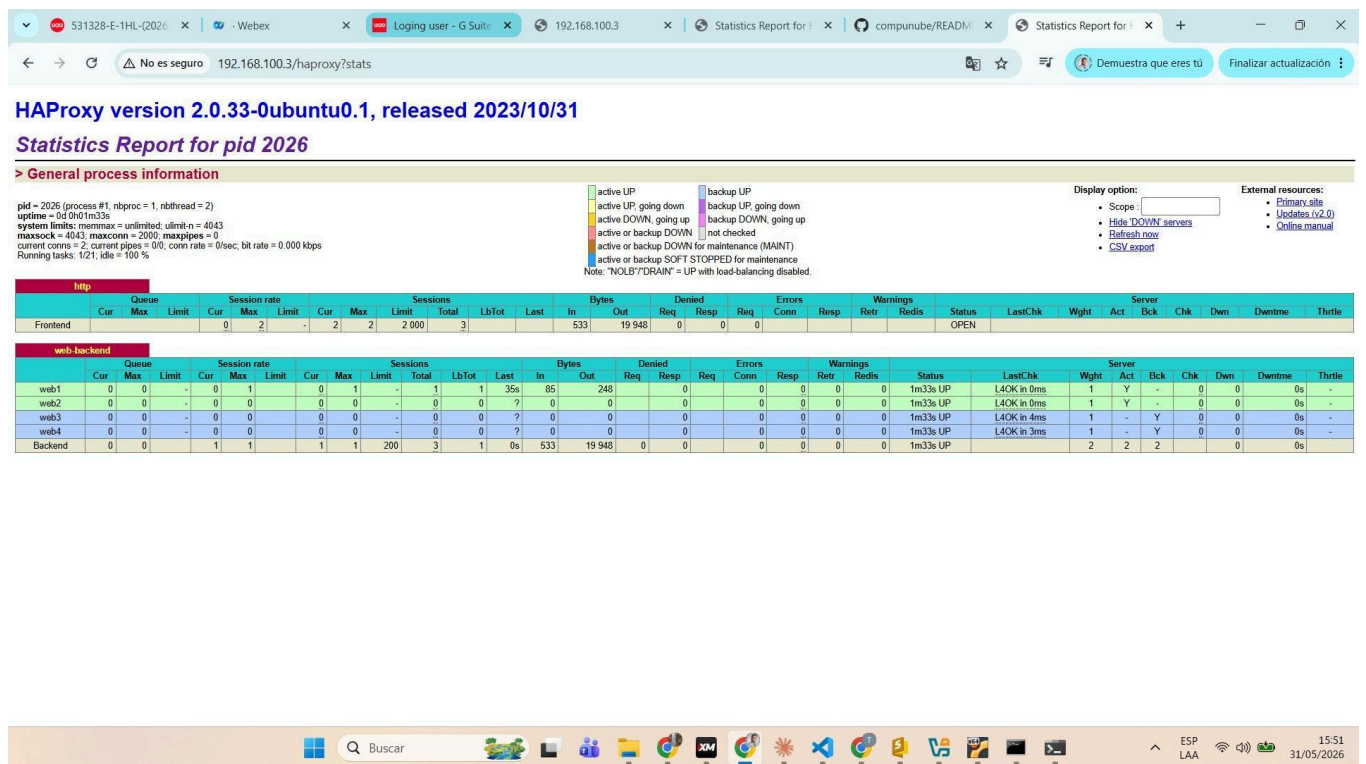


Figura 6. Dashboard HAProxy con produccion activa y backup en espera — todos UP

4.7 Pruebas de Tolerancia a Fallos

Se apagaron los contenedores web1 y web2 con `lxc stop web1` y `lxc stop web2`. HAProxy detecto la caida de los nodos activos mediante los health checks HTTP y activo automaticamente web3 y web4 como servidores de respaldo, redirigiendo todo el trafico hacia clienteUbuntu sin intervencion manual. El dashboard mostro web1 y web2 en rojo (DOWN) y web3/web4 ahora sirviendo peticiones activamente. Al apagar tambien los servidores backup, HAProxy retorno la pagina de error 503 personalizada con identidad UAO.

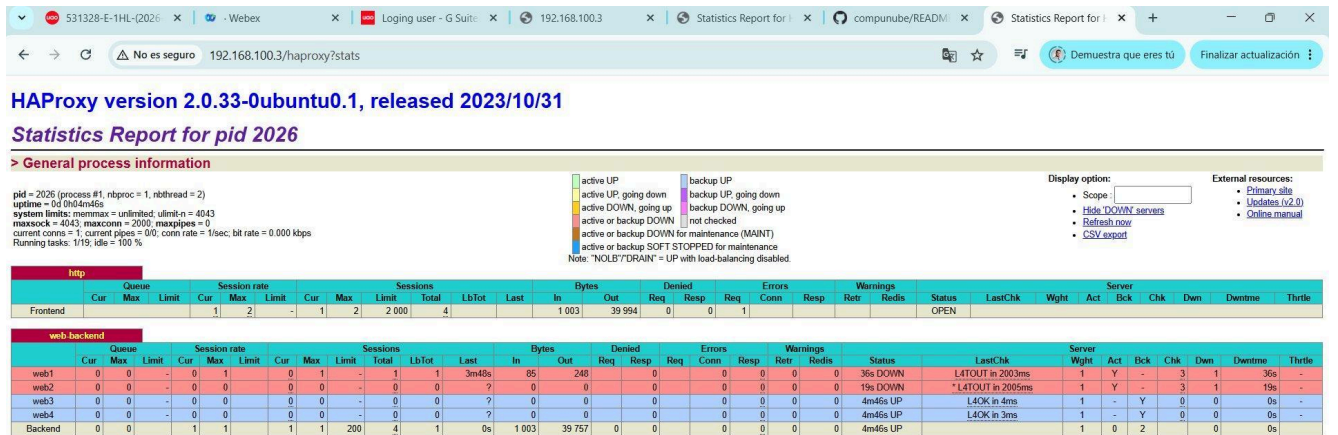


Figura 7. HAProxy con produccion caida — backup web3/web4 activados automaticamente

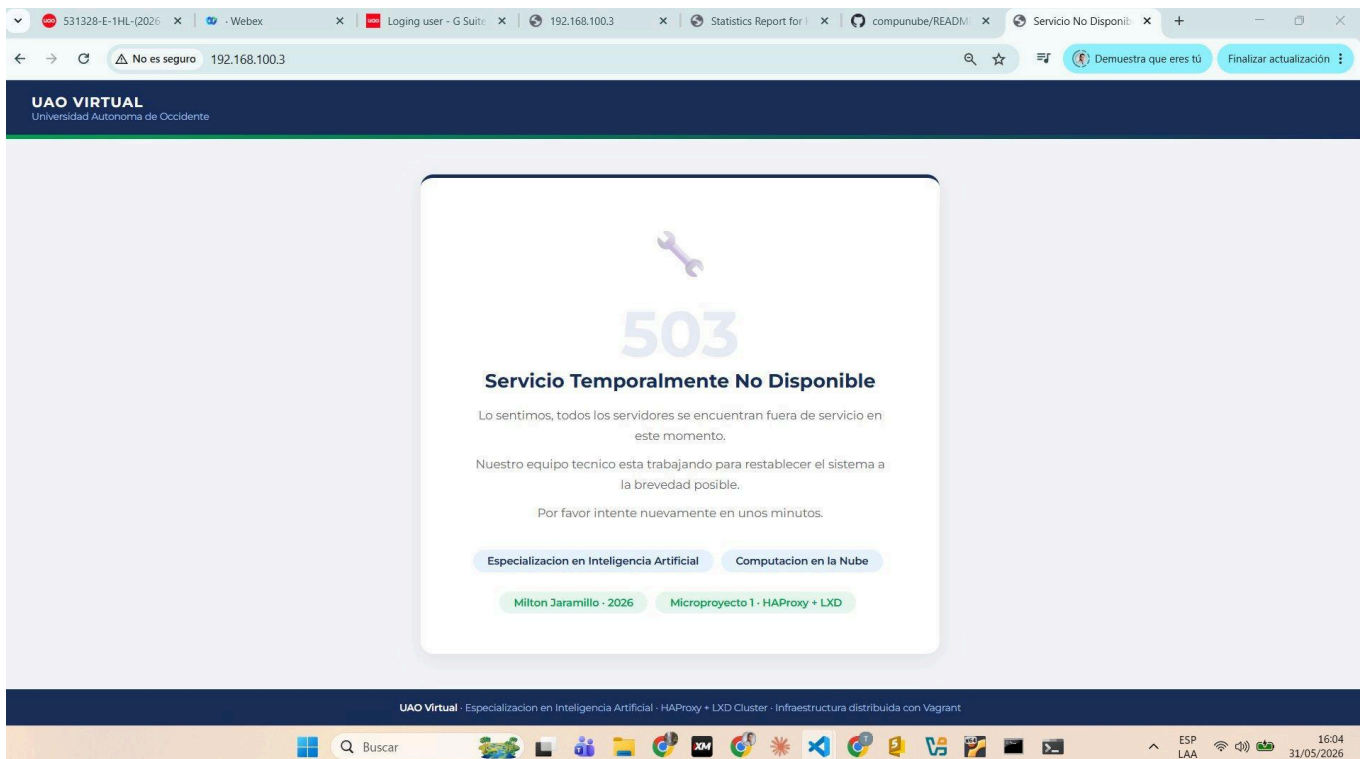


Figura 8. Pagina 503 personalizada UAO cuando todos los servidores estan caidos

5. Resultados y Analisis de Pruebas JMeter

5.1 Escenario Normal

El primer escenario simulo condiciones de operacion habitual con 100 usuarios virtuales, ramp-up de 10 segundos y 10 loops por usuario, generando 1,000 peticiones HTTP totales al balanceador en 192.168.100.3. Los resultados obtenidos fueron: throughput de 104.1 req/seg, tiempo de respuesta promedio de 39ms, tiempo minimo de 4ms, tiempo maximo de 324ms, desviacion estandar de 45.11ms y tasa de error del 0.00%. Estos resultados evidencian que bajo carga normal el sistema responde de manera optima, distribuyendo eficientemente las peticiones entre web1 y web2 sin ningun fallo.

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/s...	Sent KB/sec	Avg. Bytes
HTTP Request	1000	39	4	324	45.11	0.00%	104.1/sec	385.93	11.99	3797.0
TOTAL	1000	39	4	324	45.11	0.00%	104.1/sec	385.93	11.99	3797.0

☐ Include group name in label? ☒ Save Table Header

Figura 9. JMeter Summary Report — Escenario normal: 1,000 peticiones, 0.00% error, 39ms promedio

5.2 Escenario de Estres

El segundo escenario sometio al sistema a condiciones extremas con 500 usuarios virtuales, ramp-up de 1 segundo y 22 loops, generando 11,000 peticiones HTTP totales. Los resultados fueron: throughput de 67.6 req/seg, tiempo de respuesta promedio de 891ms, tiempo minimo de 4ms, tiempo maximo de 5,545ms, desviacion estandar de 740.25ms y tasa de error del 6.24%. La caída del throughput de 104.1 a 67.6 req/seg y el incremento del tiempo promedio de 39ms a 891ms reflejan la saturacion progresiva de los recursos del sistema bajo carga extrema. La tasa de error del 6.24% indica que una fraccion de las peticiones supero el timeout de conexion configurado en HAProxy.

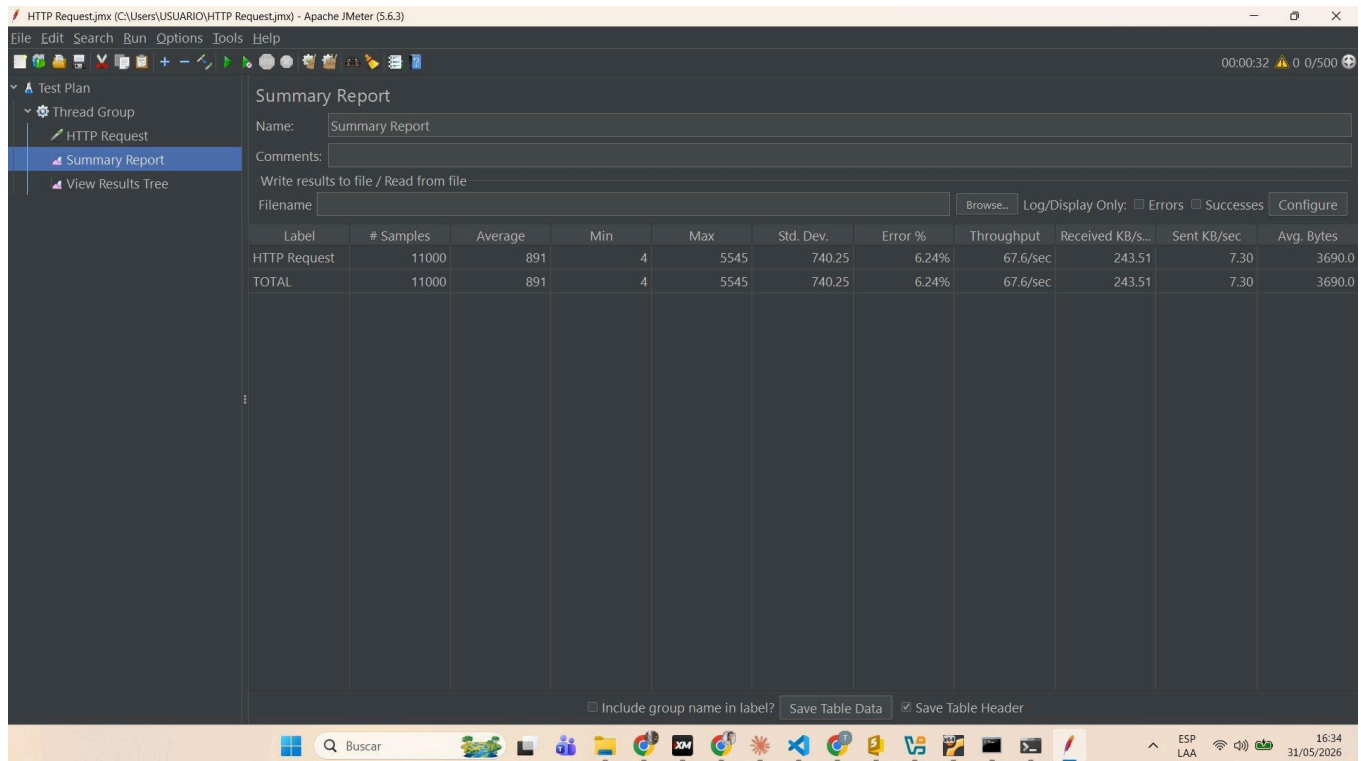


Figura 10. JMeter Summary Report — Escenario estres: 11,000 peticiones, 6.24% error, 891ms promedio

5.3 Analisis Comparativo

La comparacion entre ambos escenarios revela el comportamiento del sistema bajo diferentes niveles de carga. Con carga normal (100 usuarios), el balanceador opero sin errores, con latencias bajas y throughput estable. Al incrementar a 500 usuarios simultaneos, el throughput cayo un 35% y la latencia promedio se multiplico por 23, lo que indica que el cuello de botella se encuentra en la capacidad de procesamiento de los contenedores Apache mas que en el propio balanceador. La alta desviacion estandar (740ms) en el escenario de estres confirma una distribucion muy dispersa de los tiempos de respuesta, con algunos requests atendidos en 4ms y otros tardando hasta 5.5 segundos, tipico de sistemas bajo saturacion parcial.

6. Conclusiones

La implementacion del cluster LXD multi-nodo demostro que es posible crear infraestructura distribuida de alta disponibilidad sobre recursos de hardware modestos (dos VMs Vagrant), replicando patrones arquitectonicos utilizados en entornos cloud empresariales. La capacidad de distribuir contenedores entre nodos del cluster con la directiva `--target` y gestionarlos desde un punto unico de control es una ventaja significativa de LXD frente a soluciones de contenedores mas simples.

El mecanismo de servidores backup en HAProxy demostro ser una solucion robusta y completamente automatica para el failover. La transicion de produccion a backup se realizo sin interrupcion perceptible del servicio, activandose unicamente cuando el health check HTTP

confirmando la indisponibilidad de todos los nodos activos. Este comportamiento es esencial en arquitecturas de alta disponibilidad donde el tiempo de recuperación (RTO) debe ser mínimo.

El reto de comunicación cross-node entre contenedores en diferentes nodos del cluster fue resuelto de manera elegante mediante proxy devices LXD, sin necesidad de modificar la red del cluster ni instalar software adicional. Esta técnica es transferible a cualquier escenario donde se requiera exponer servicios de contenedores a redes externas de forma controlada.

Los resultados de JMeter evidencian que el sistema mantiene un rendimiento óptimo bajo carga normal (0% errores, 39ms promedio) pero comienza a degradarse con 500 usuarios concurrentes (6.24% errores, 891ms promedio). Para escalar este sistema en producción, se recomienda incrementar los recursos de las VMs o agregar más nodos al cluster para distribuir mejor la carga entre contenedores Apache adicionales.

7. Referencias

HAProxy Technologies. (2026). HAProxy — The Reliable, High Performance TCP/HTTP Load Balancer. Recuperado de <http://www.haproxy.org/>

Linux Containers. (2026). LXD Clustering. Recuperado de <https://linuxcontainers.org/lxd/docs/master/clustering/>

Apache Software Foundation. (2026). Apache JMeter. Recuperado de <https://jmeter.apache.org/>

Martínez Velásquez, H. F. (2026). Microproyecto 1 — Cluster LXD + HAProxy con Servidores Backup. UAO Virtual, Computación en la Nube, Especialización en Inteligencia Artificial.

Jaramillo, M. (2026). Repositorio compunube. GitHub. Recuperado de <https://github.com/milton329/compunube>

HashiCorp. (2026). Vagrant Documentation. Recuperado de <https://www.vagrantup.com/>